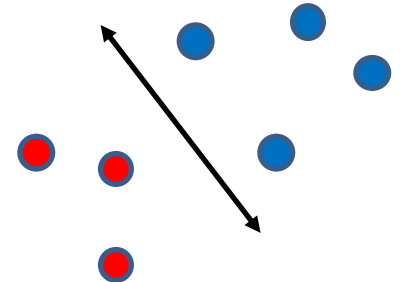


Clarkson's algorithms

Lee-Ad Gottlieb

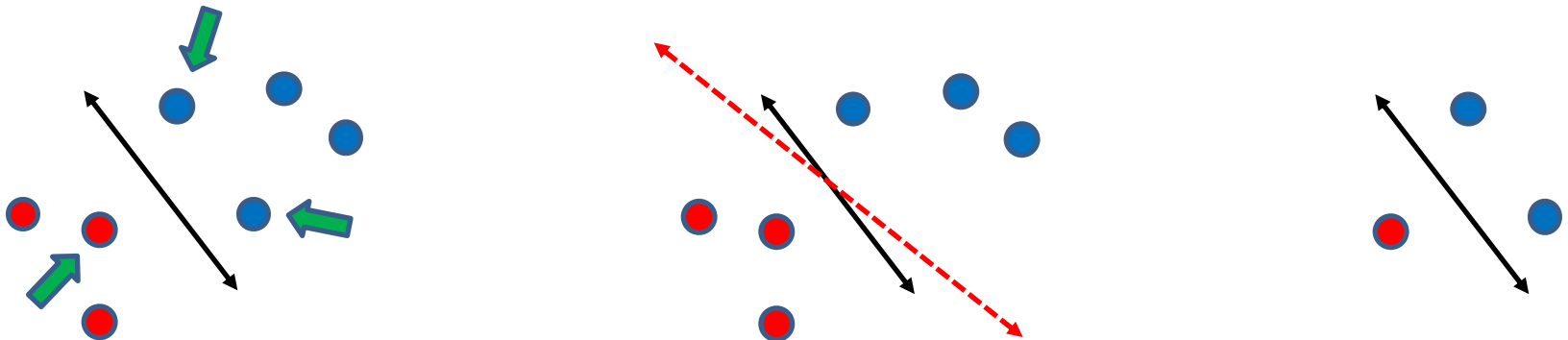
Hyperplanes

- Perceptron:
 - Runtime: $O(1/\gamma^2)$, expensive when γ is small
 - Doesn't return max-margin hyperplane



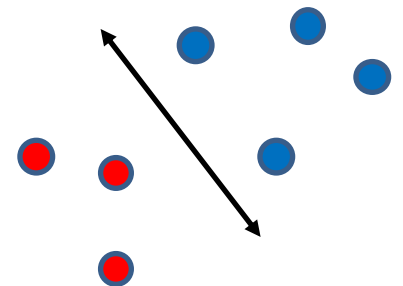
Hyperplanes

- How expensive is it to find the max-margin hyperplane separator?
 - Let's try brute force
- Definition: The **Support Vectors** of S are those which define the max-margin of S
 - If even one of them is removed, a bigger margin is possible (assuming no ties)
 - Any subset of S that contains the Support Vectors has the same max-margin
- In d -dimensional space, at most $d+1$ support vectors



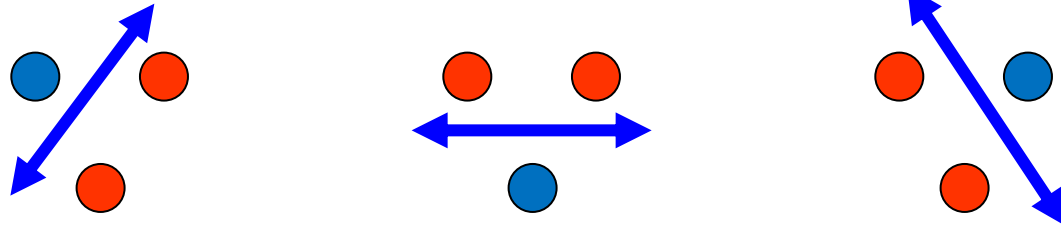
Hyperplanes

- How expensive is it to find the max-margin hyperplane separator?
- Brute force:
 - Try all combinations of $d+1$ points out of n points in S . One combination has all support vectors
 - Runtime: $n^{O(d)}$
 - Finds max-margin hyperplane
- Can we improve on the runtime?
 - Clarkson's algorithms

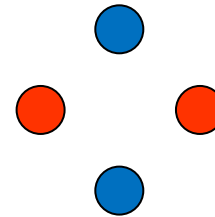


Preliminaries

- VC dimension
 - A line can **shatter** 3 points



A line cannot **shatter** any collection of 4 points
So a line has VC dimension of **3**



- More generally, a d -dimensional hyperplane has VC dimension: **$d+1$** .

Clarkson

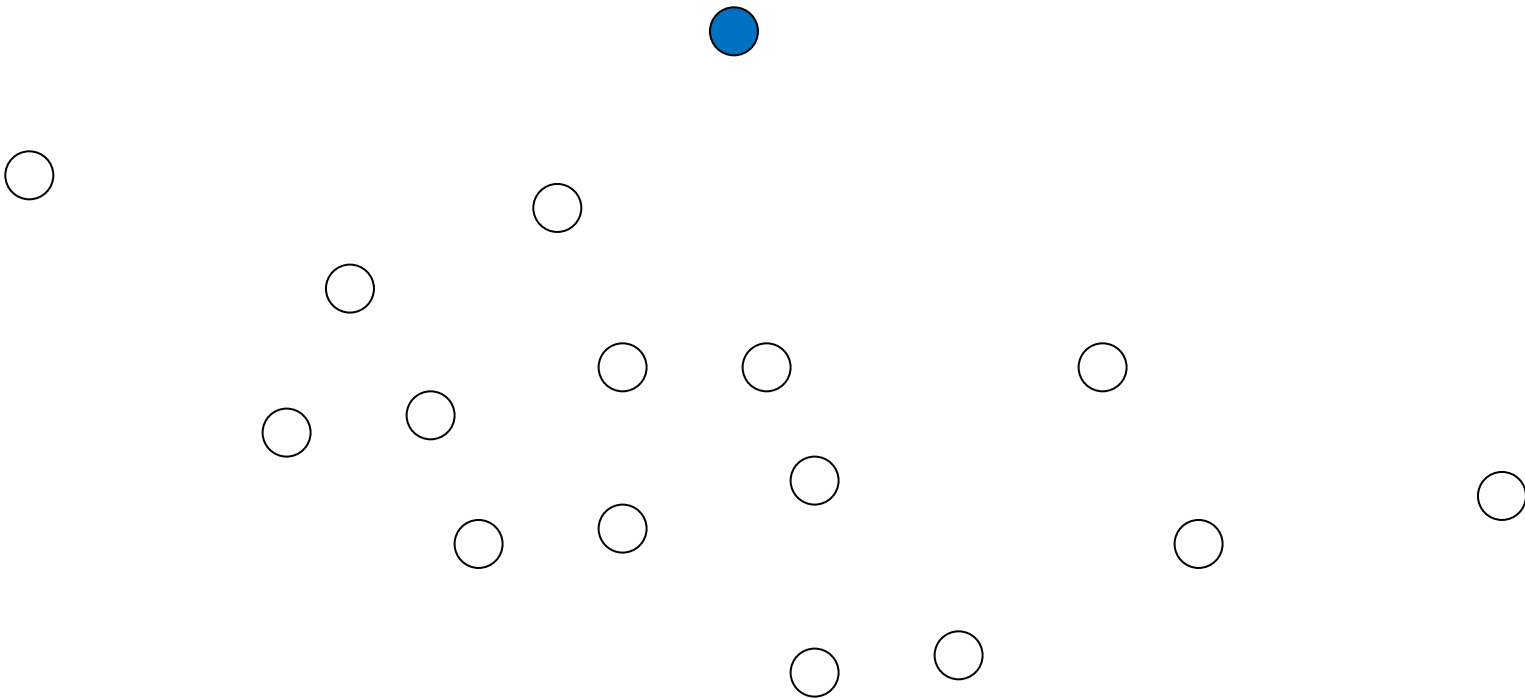
- Clarkson gave two algorithms
- They break down the problem into smaller sub-problems:
 - Let **P** be an algorithm that finds the max-margin separator.
 - Let **P(n,d)** on a set of size n with dimension d
 - For example, the brute-force algorithm has
$$P(n,d) = n^{O(d)}$$
 - Instead of running **P** on all our set, we'll run it on sub-problems of size smaller than d

Clarkson 1

- Clarkson's first algorithm
- Input: d -dimensional points set $\mathbf{S}$ of size n
 - 1. Set $\mathbf{S}' = \emptyset$
 - 2. Sample from $\mathbf{S}$ a subset $\mathbf{B}$ of size m .
 - 3. $\mathbf{S}' = \mathbf{S}' \cup \mathbf{B}$
 - 4. Run $\mathbf{P}$ on $\mathbf{S}'$ to find a max-margin separator $\mathbf{h}$ on $\mathbf{S}'$ in time $P(|\mathbf{S}'|, d)$
 - 5. If $\mathbf{h}$ is max-margin classifier on $\mathbf{S}$, exit program
 - 6. Add to $\mathbf{S}'$ points of $\mathbf{S}$ misclassified by $\mathbf{h}$
 - 7. Go to 4.
- How many iterations does the program make?

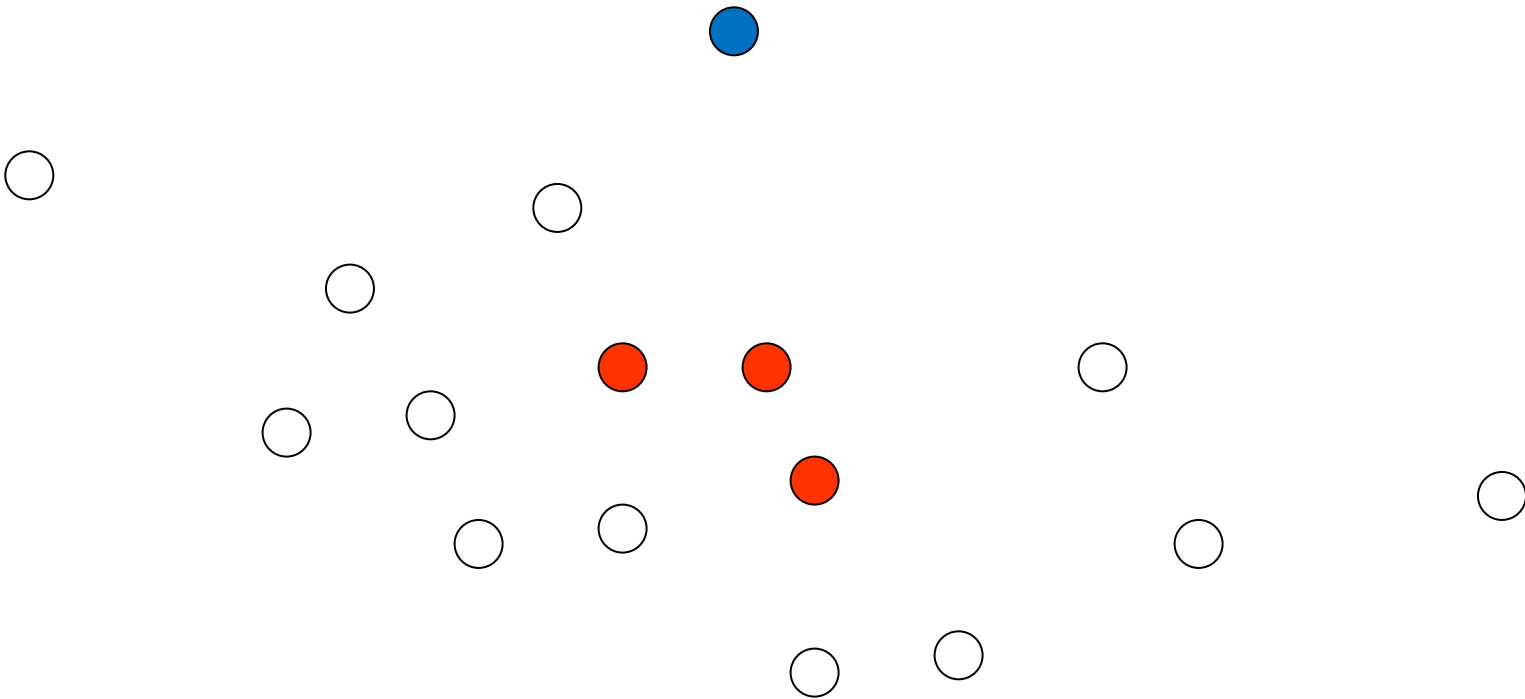
Clarkson 1

- For simplicity, assume we have only one blue point and are sampling from the reds.



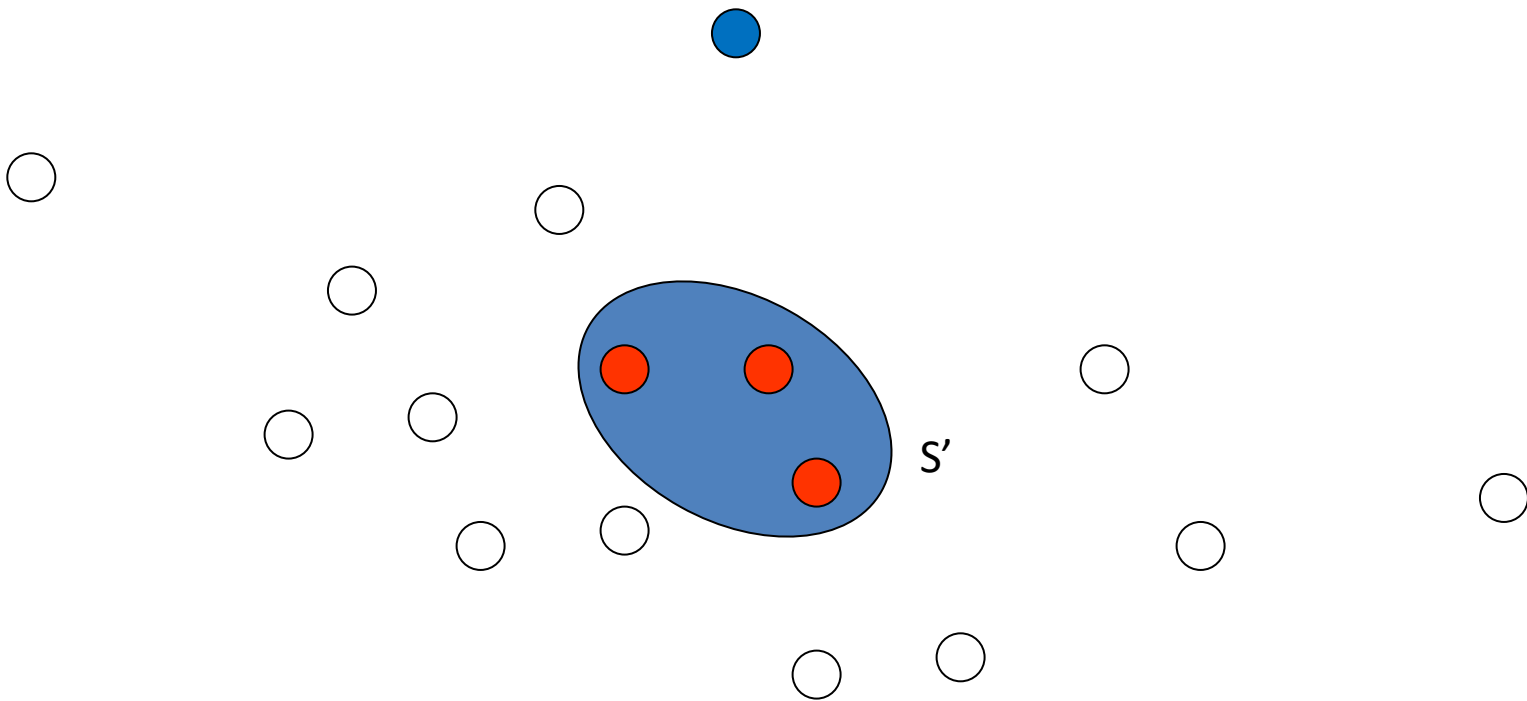
Clarkson 1

- Sample a subset B of size m



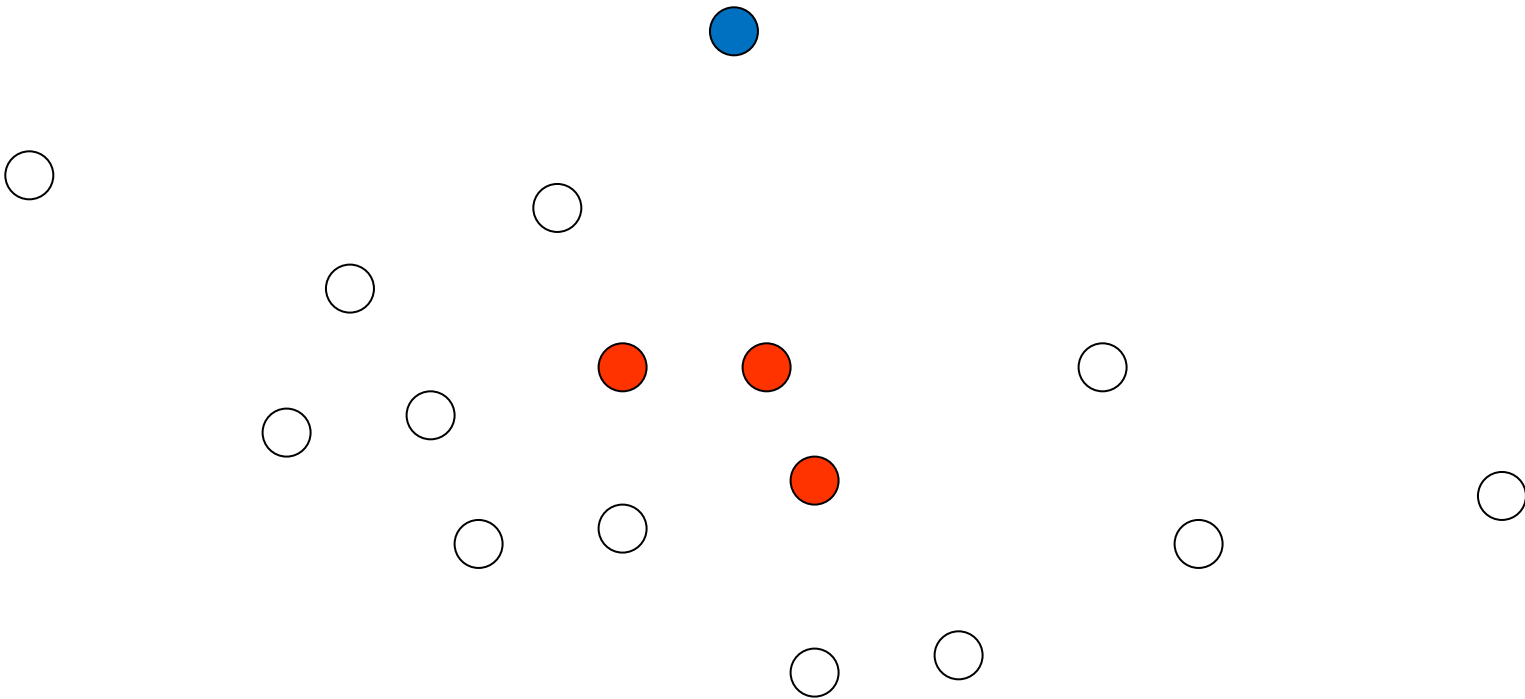
Clarkson 1

- Add B to S'



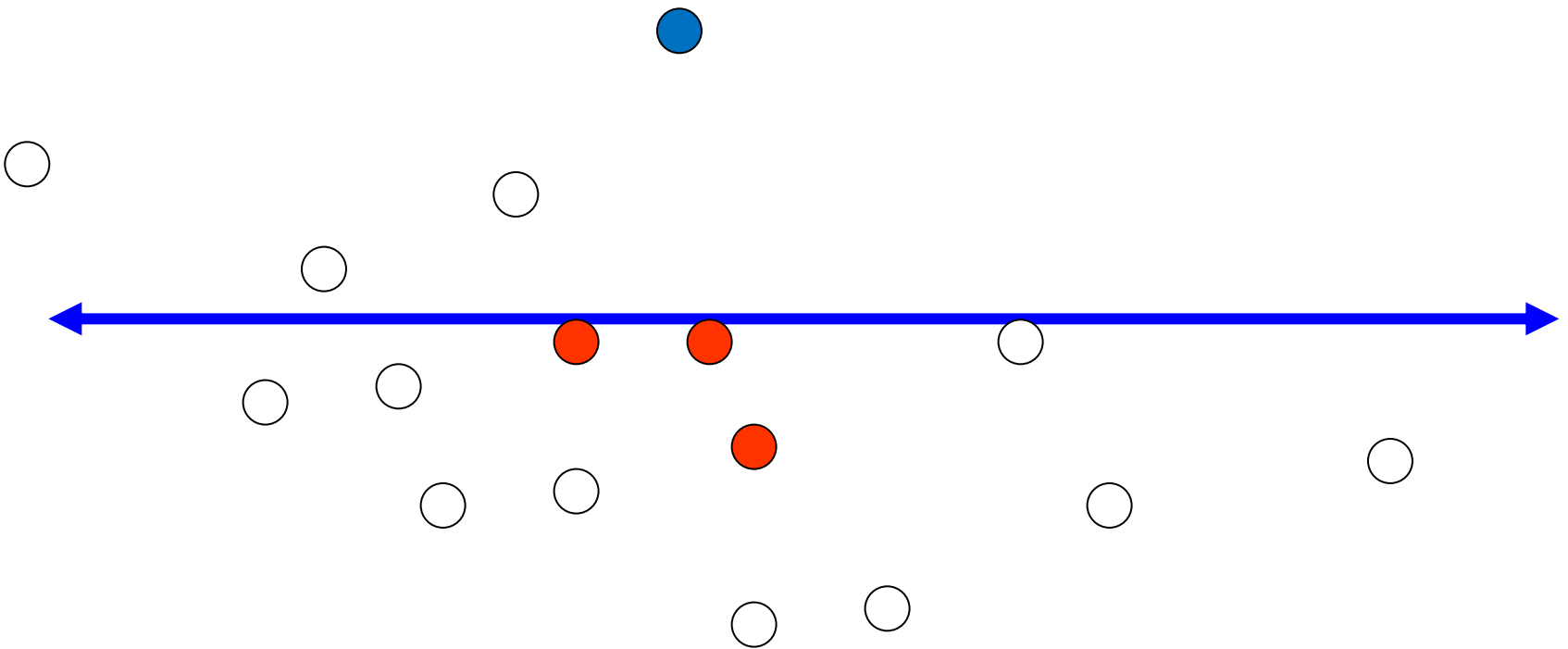
Clarkson 1

- Run P on S' in time $P(|S'|, d)$



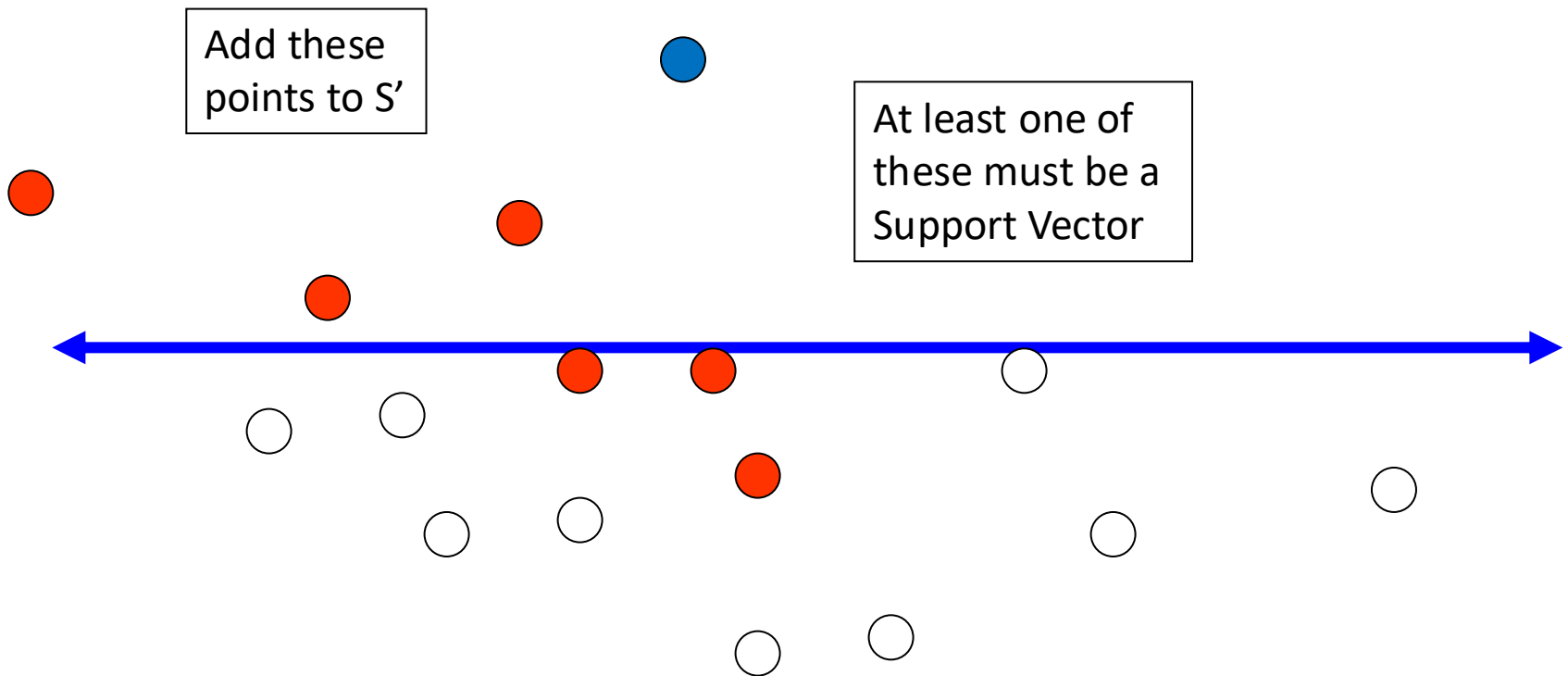
Clarkson 1

- Compute max-margin h on S'



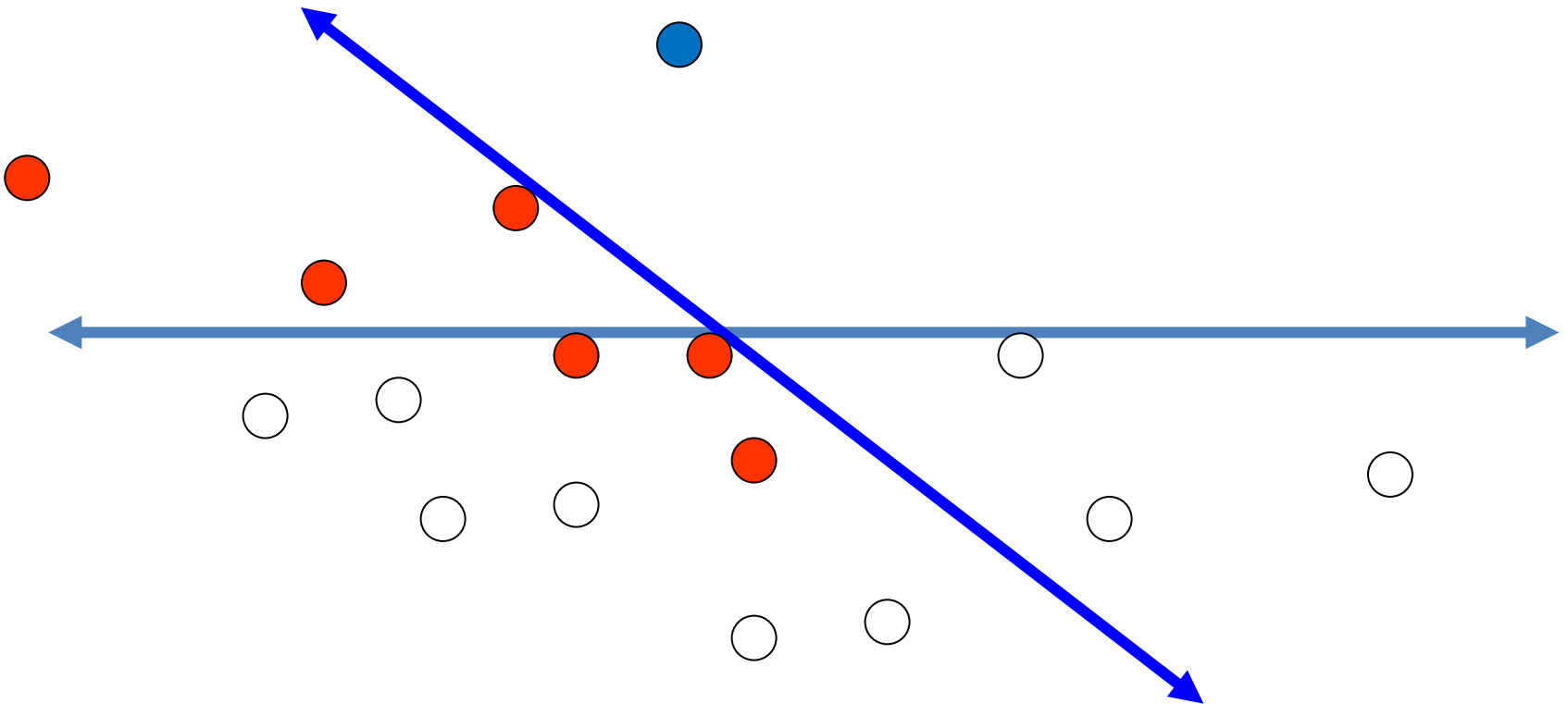
Clarkson 1

- Find points misclassified by h , add them to S'



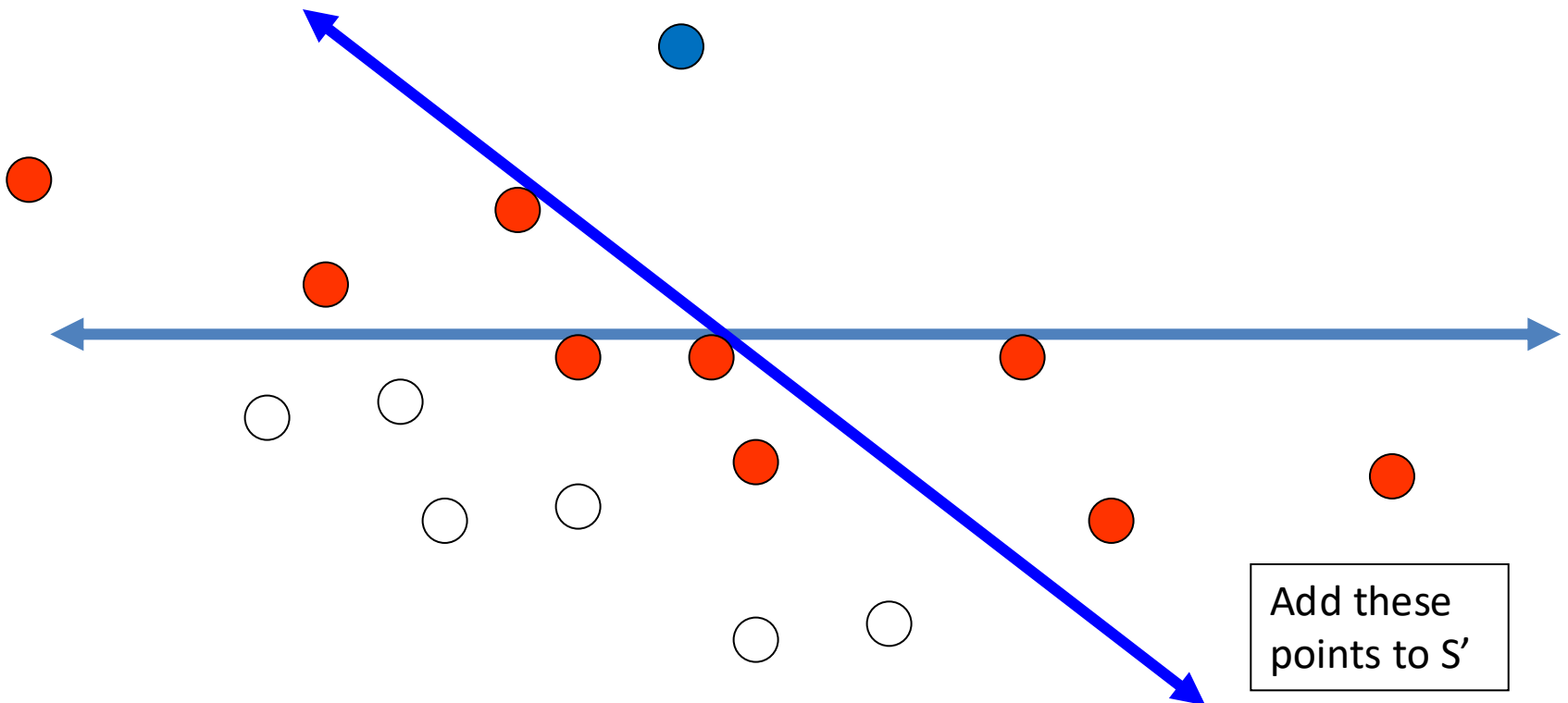
Clarkson 1

- Run P on S' , find new max-margin separator h



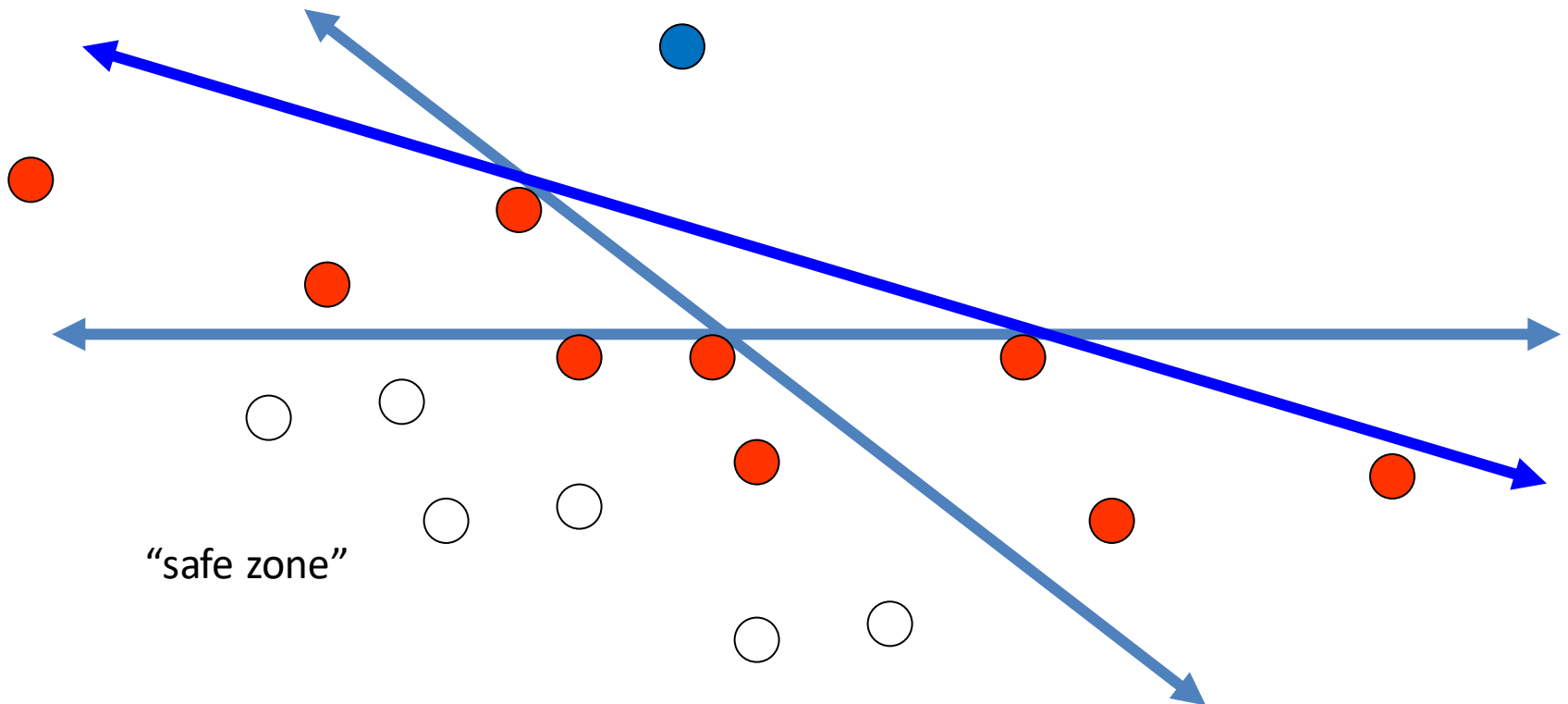
Clarkson 1

- Add misclassified points to S'



Clarkson 1

- Run P again on S' , find max-margin h , and exit



Analysis of Clarkson 1

- How many iterations does the algorithm make?
 - Claim: In every iteration, $\mathbf{h}$ is either consistent on all of $\mathbf{S}$, or misclassifies at least one Support Vector
 - Proof: By definition, a max-margin hyperplane that is consistent on all Support Vectors is consistent on all points.
- There are $d+1$ support vectors, so there are at most $d+2$ iterations.

Analysis of Clarkson 1

- How big should our samples from **S** be?
- Recall: With probability $1-1/m$,
 - any hypothesis that correctly classifies all point of an m -point sample S' , misclassifies at most

$$O\left(\frac{n}{m} (\log m + d_{VC} \log(m/d_{VC}))\right) = O^*(d_{VC} n/m)$$

points of S .

- Our hyperplanes have VC-dimension $d+1$

Analysis of Clarkson 1

- Choose $m = O(dn^{.5})$
- At each iteration, h misclassifies
$$O(d_{VC}n/m) = O(dn / dn^{.5}) = O(n^{.5})$$

points.
- After $d+2$ iterations, $|S'| = O(dn^{.5})$

Analysis of Clarkson 1

- Synopsis:
 - C1 breaks the problem down into **d** computations of **$P(dn^5, d)$** .
 - For each hyperplane, did **dn** work.
 - **$P(n, d) = O(d^2n) + d P(dn^5)$**
- If P is brute force algorithm, this gives a runtime of about $d^3n^{d/2}$ instead of about n^d .

Clarkson 2

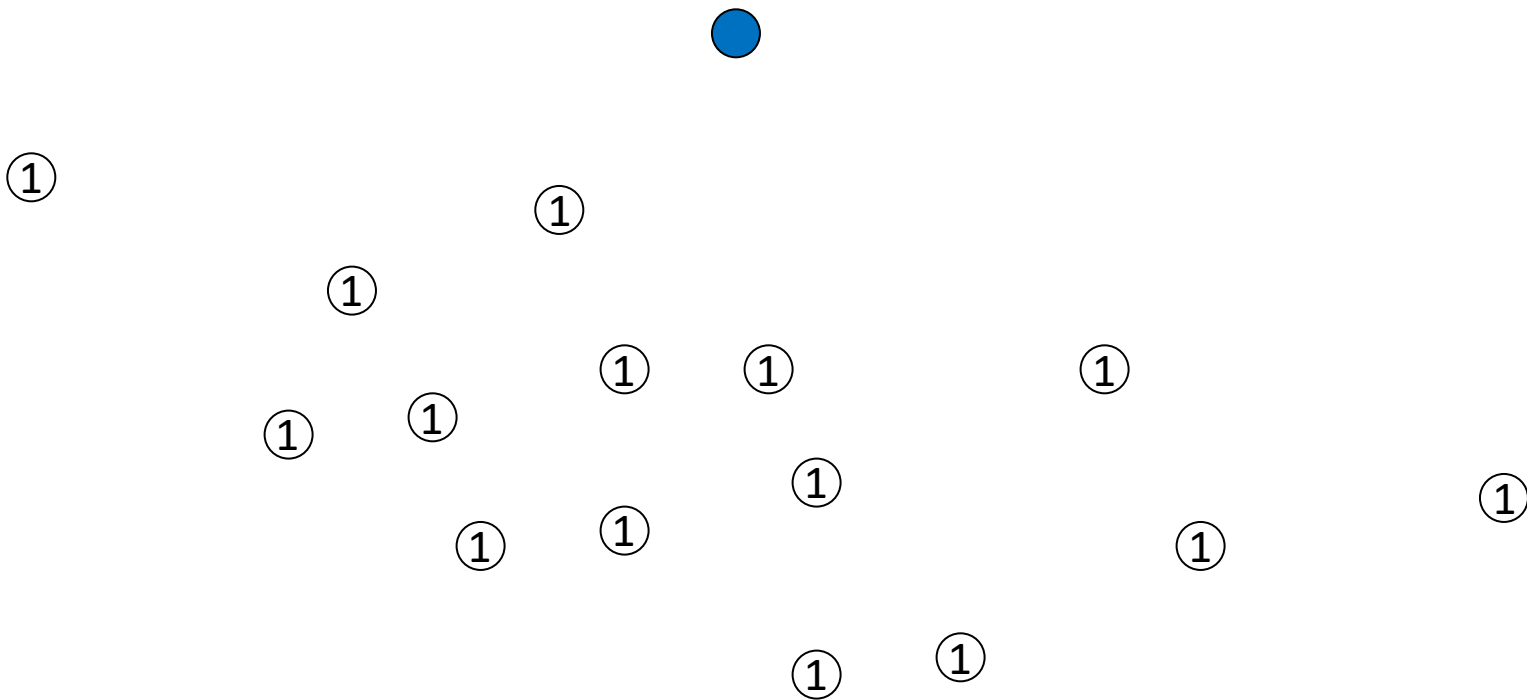
- Clarkson's second algorithm uses iterative reweighting.
 - Multiple iterations of sampling with replacement
 - Each iteration produces a hyperplane
 - Each point has a *weight*. Think of this as copies of the point.
- Let's take a look...

Clarkson 2

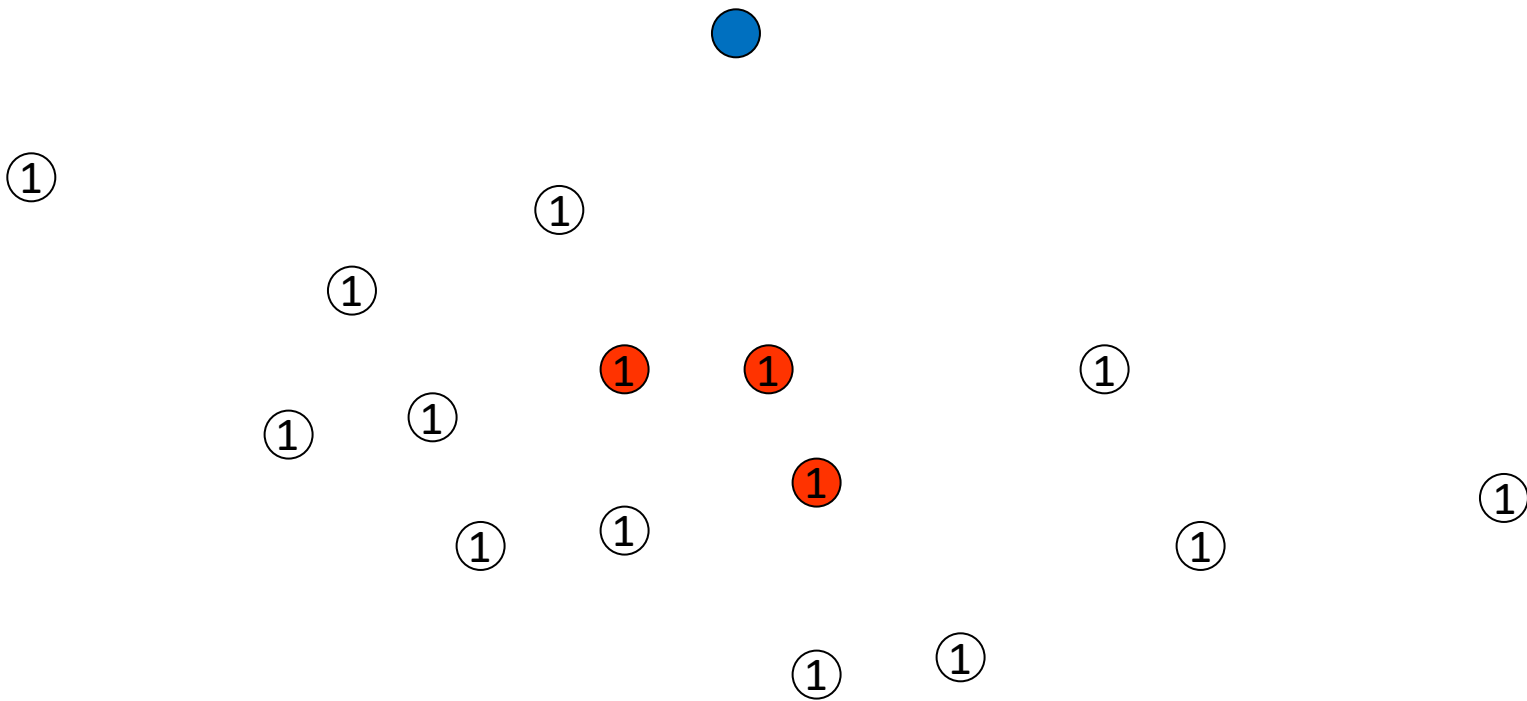
- Clarkson's second algorithm
- Input: d -dimensional point set $\mathbf{S}$ of size n . c is some constant
 - 1. Set weight $w(x) \leftarrow 1$ for every $x \in S$
 - 2. Sample from $\mathbf{S}$ *by weight* a subset $\mathbf{B}$ of size cd^2
 - 3. Run $\mathbf{P}$ on $\mathbf{B}$ to find a max-margin separator $\mathbf{h}$ on $\mathbf{B}$ in time $P(cd^2, d)$
 - 4. If $\mathbf{h}$ is consistent on $\mathbf{S}$, exit program
 - 5. For all $x \in S$ misclassified by $\mathbf{h}$, $w(x) \leftarrow 2w(x)$
 - 6. Go to 2.
- How many iterations does the program make?

Clarkson 2

- Sample a subset B of size m



Clarkson 2



Clarkson 2

At least one
of these points
Must be a SV



①

①

①

①

①

①

①

①

①

①

①

①

①

①

①



Clarkson 2

Reweight
these
points



②

②

②



①

①

①

①

①

①

①

①

①

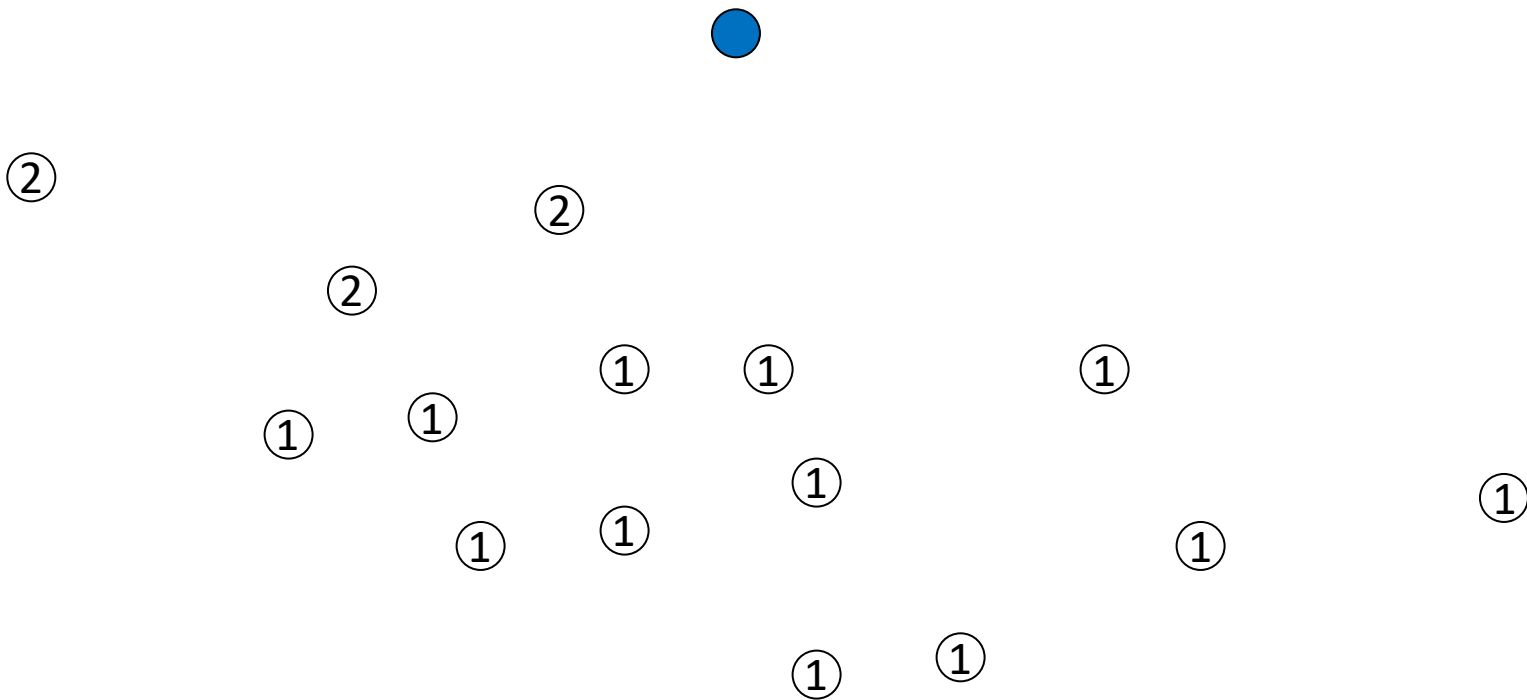
①

①

①

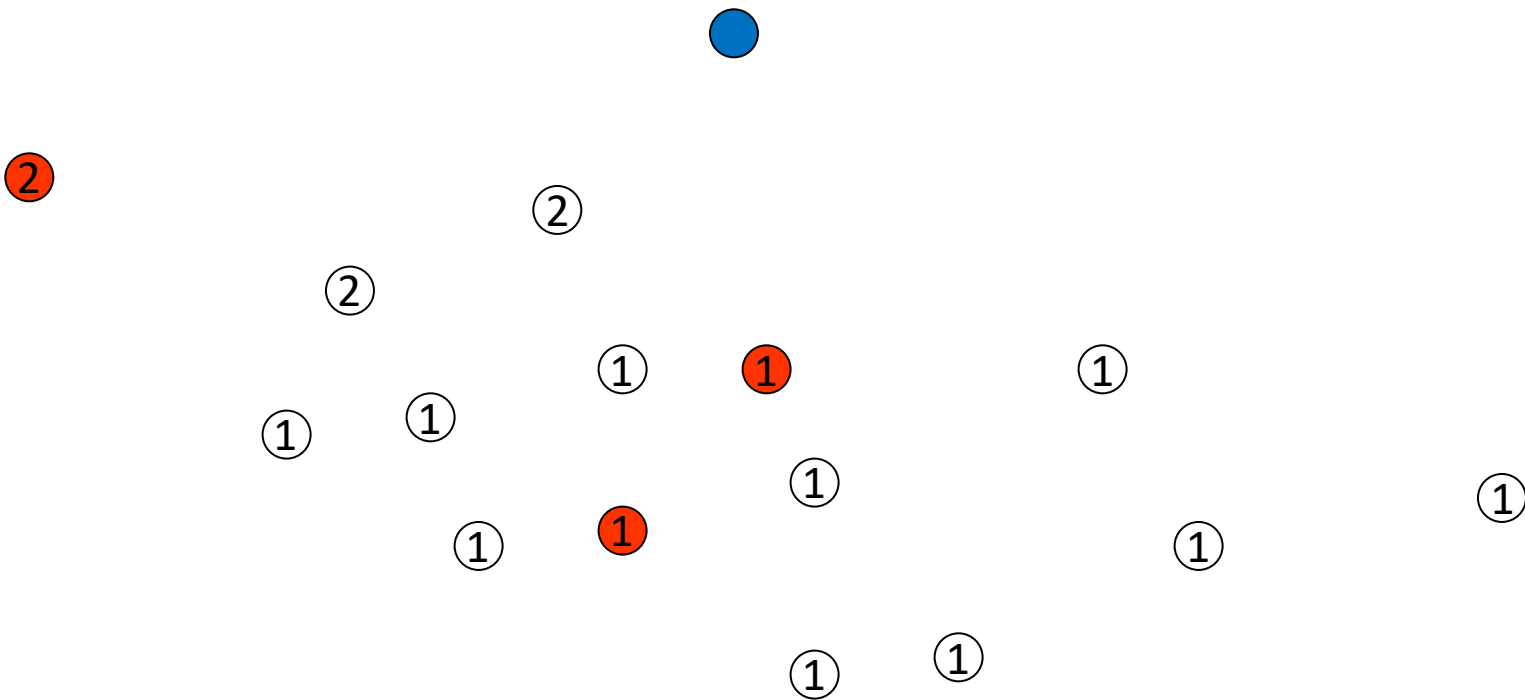
Clarkson 2

- Next iteration



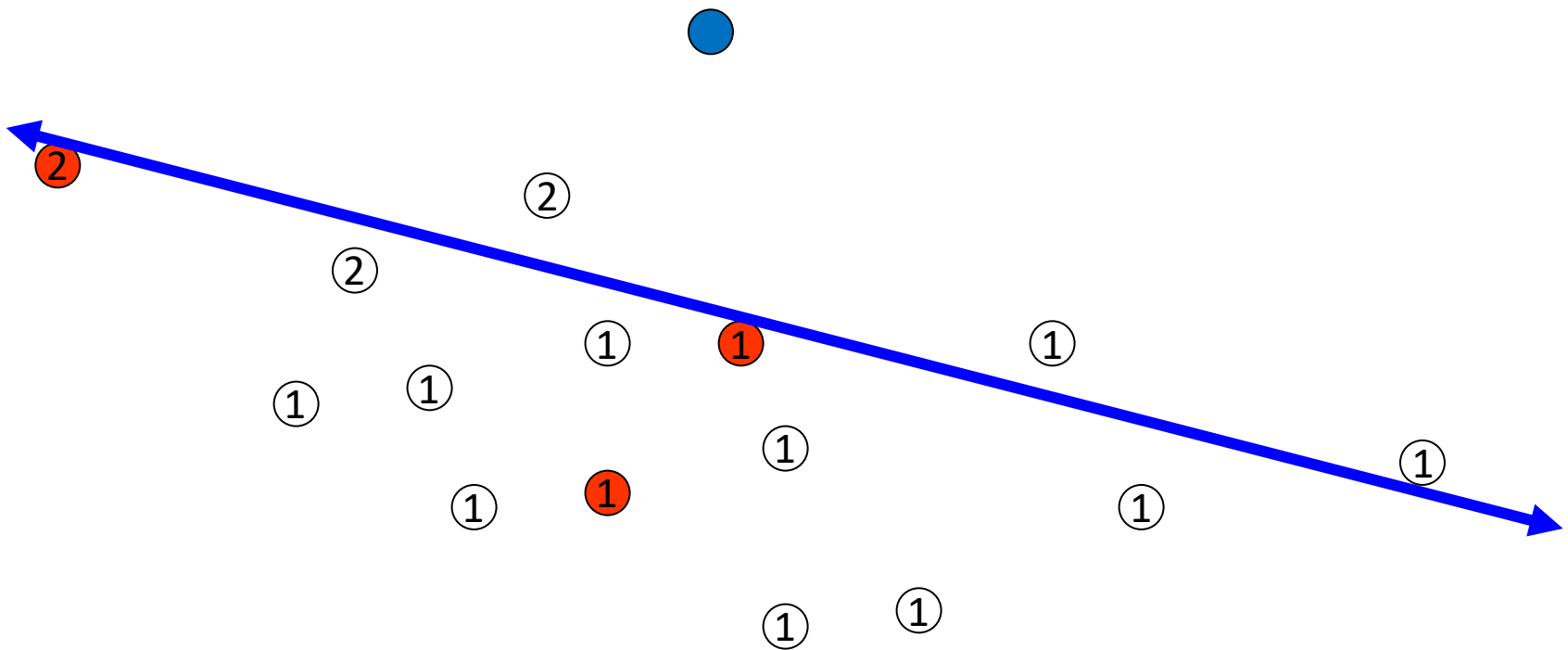
Clarkson 2

- Sample again a subset B of size m



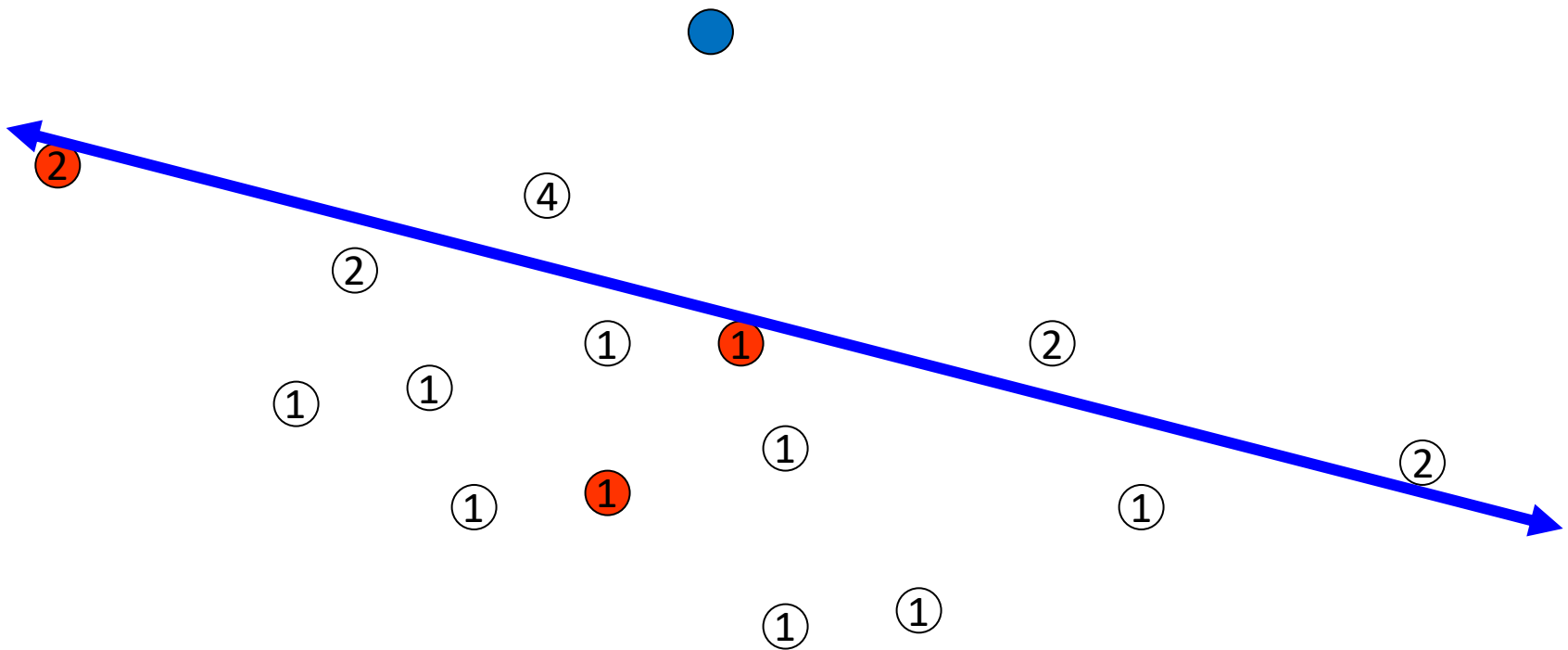
Clarkson 2

- Compute max-margin separator



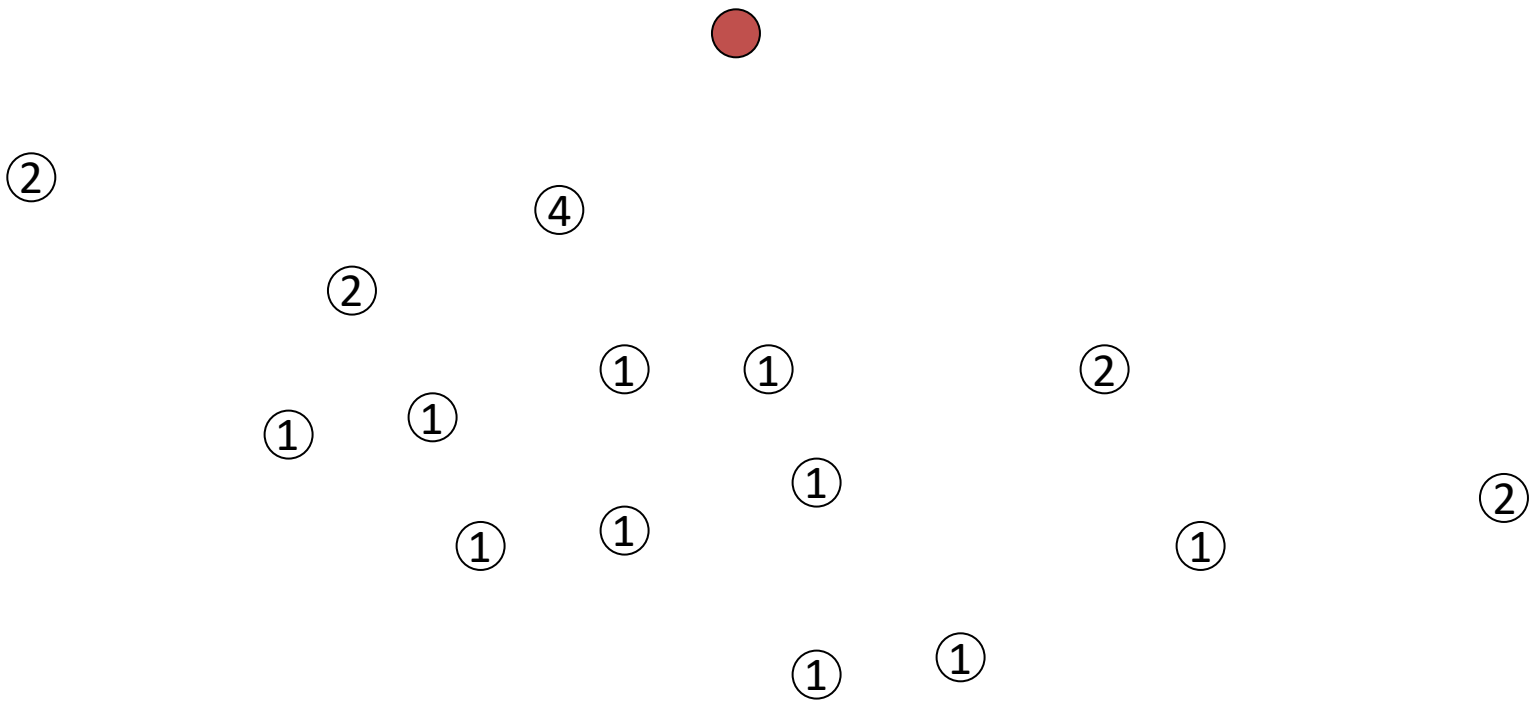
Clarkson 2

- Double the weight of misclassified points



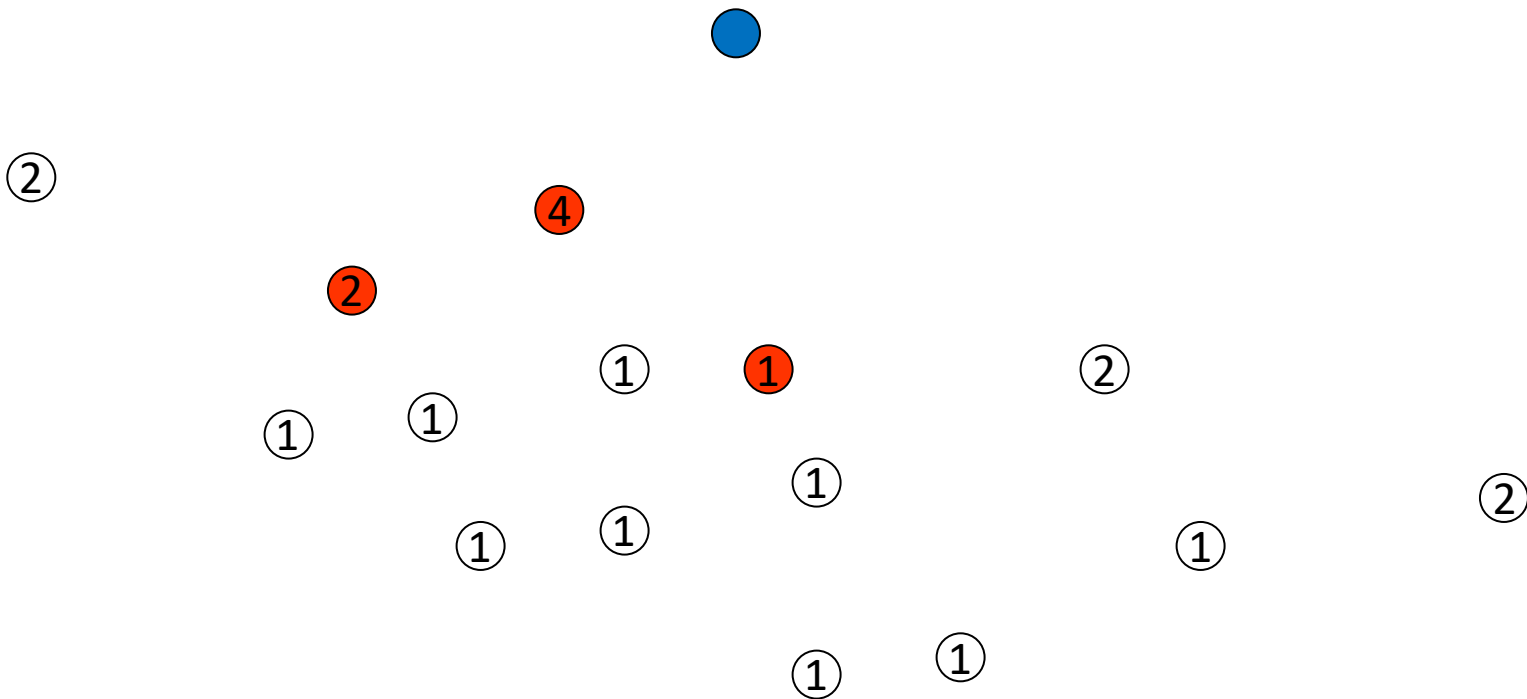
Clarkson 2

- Next iteration



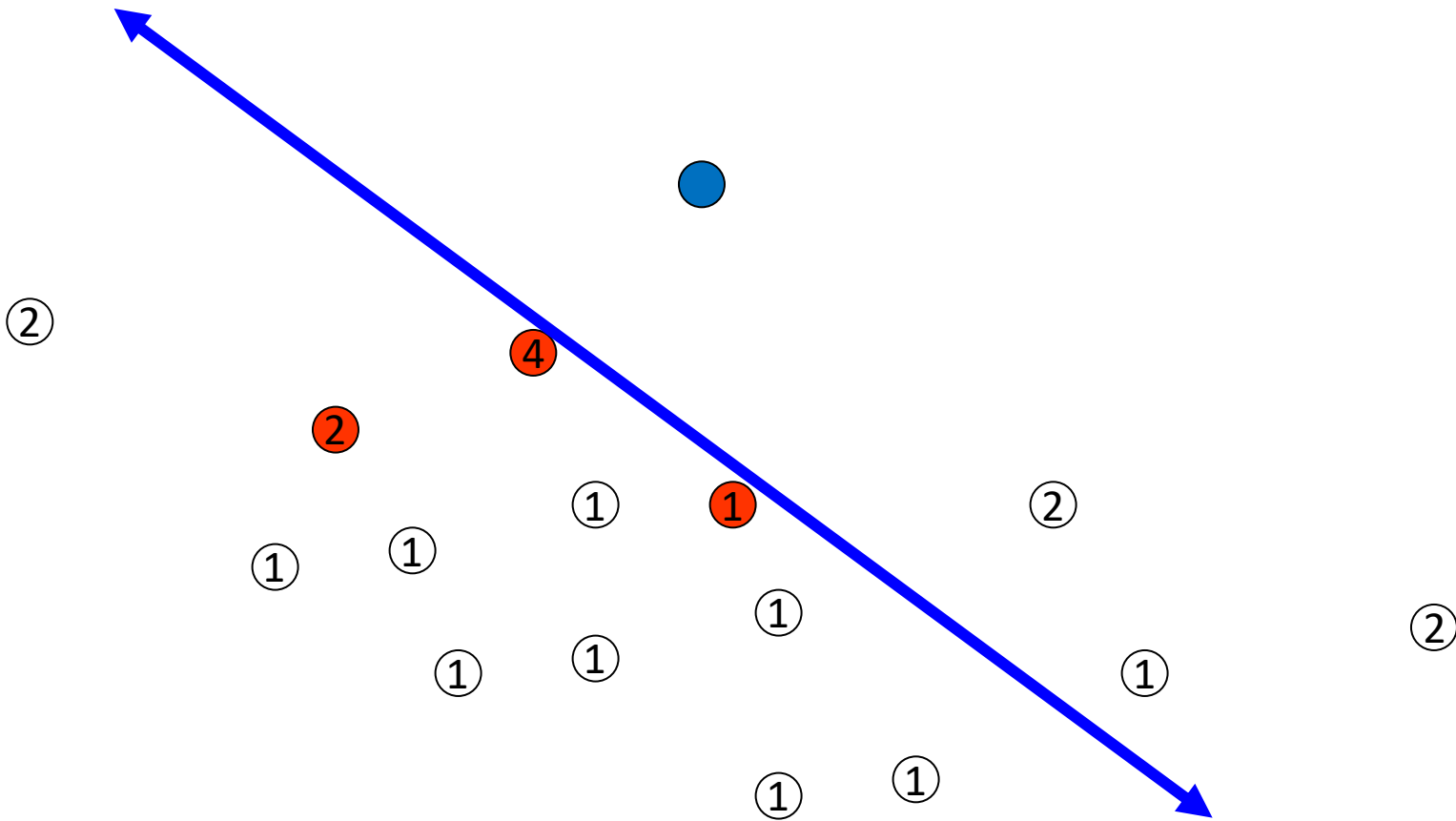
Clarkson 2

- Sample a subset B of size m



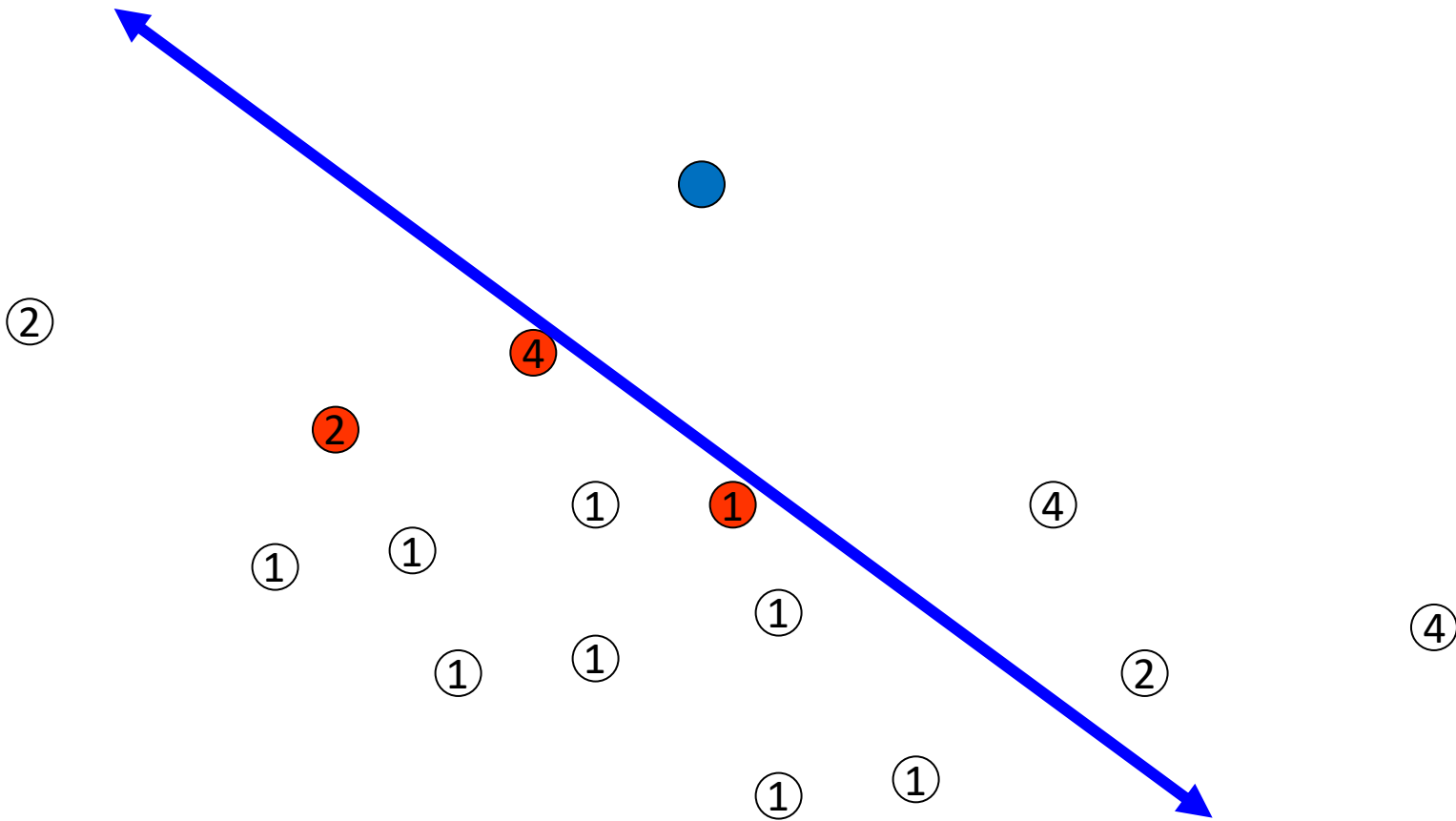
Clarkson 2

- Compute max-margin separator



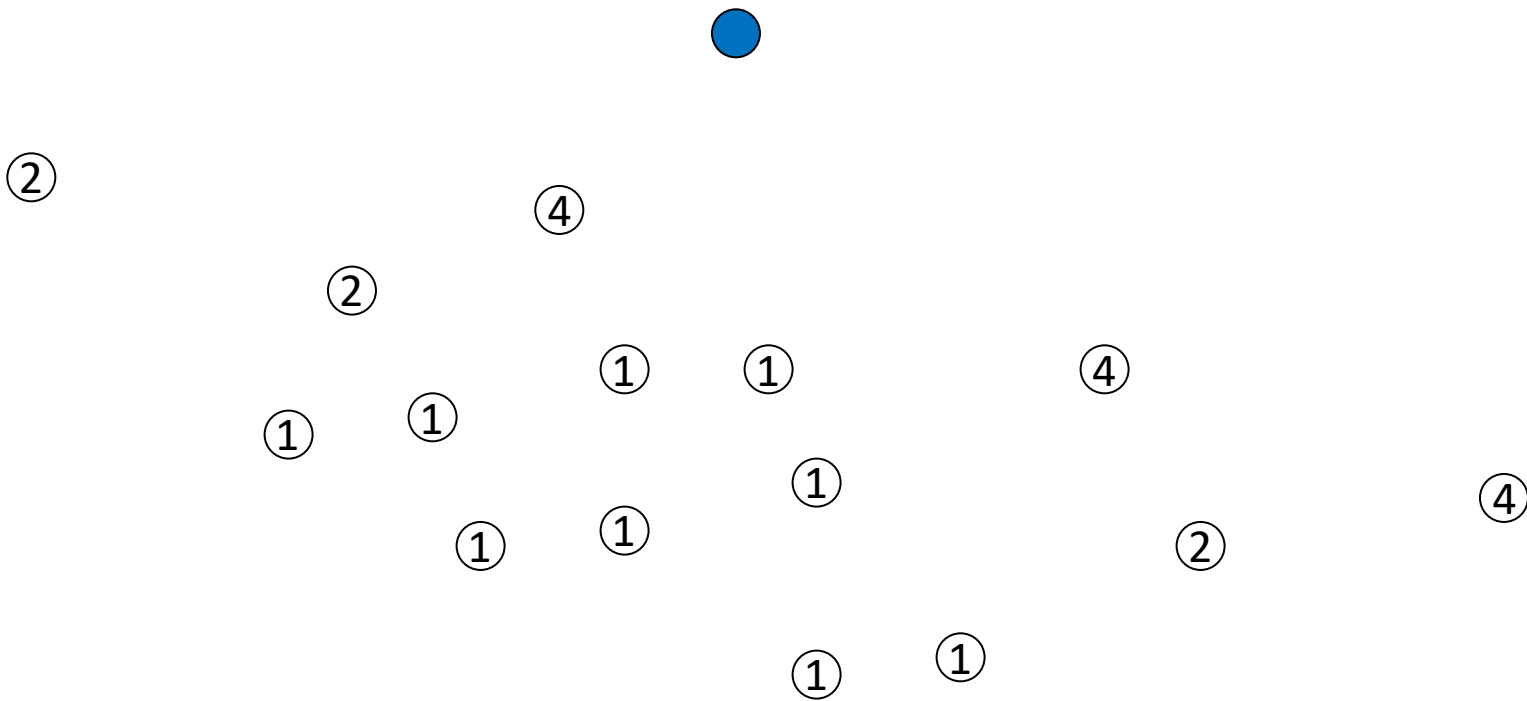
Clarkson 2

- Double the weight of misclassified points



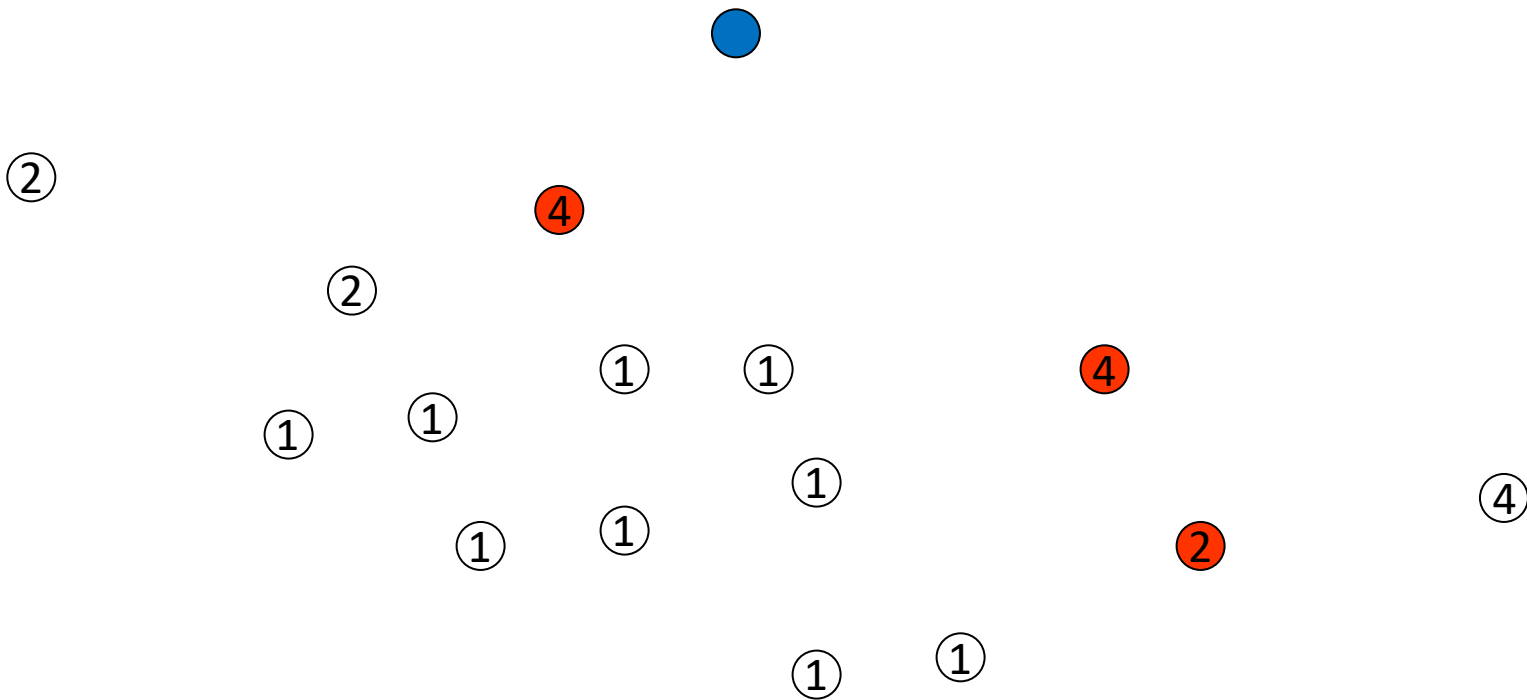
Clarkson 2

- Next iteration



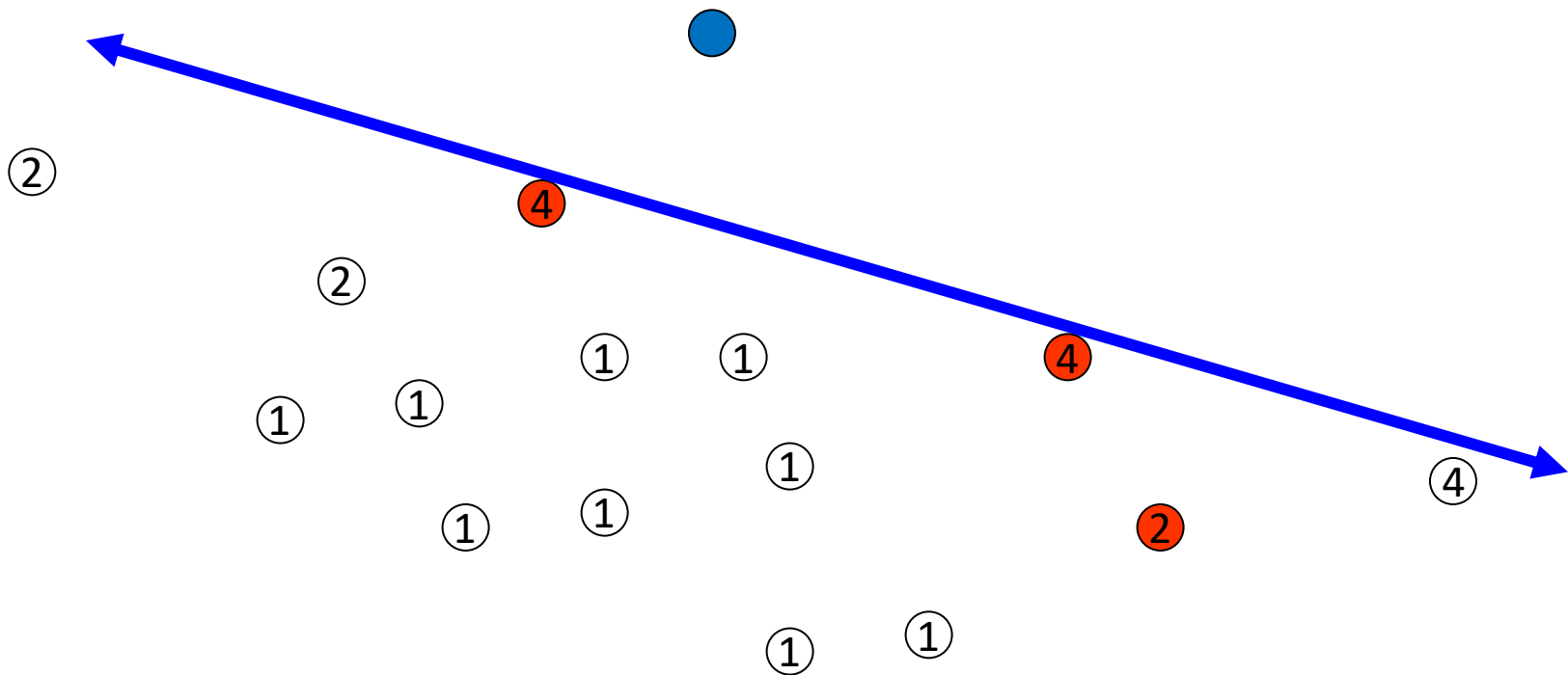
Clarkson 2

- Sample a set B of m points



Clarkson 2

- Compute max-margin separator
- Consistent with all of S – exit!



Analysis of Clarkson 2

- Quite surprisingly, this actually works.
- We need only two simple facts to prove correctness. (next slide)
- Important caveat:
 - If the weight of points misclassified by h is very heavy ($2n/cd$ or more), complete iteration and resample.
 - Chance of this happening is small:
$$O(d_{vc}n/m) = O(dn / d^2) = O(n/d)$$

Analysis of Clarkson 2

- Fact 1
 - At each iteration, the weight of at least one SV is doubled.
 - After $d+1$ iterations, the weight of the set of SV at least doubles.
 - After $k(d+1)$ iterations, the weight of the set of SV is at least $2^k(d+1)$.
- Fact 2
 - At each iteration, the weight of the entire set increases by at most $2w(S)/cd$
 - After kd iterations, the weight of the entire set is at most $(1+2/cd)^{kd}n < e^{2k/c}n$.
- The weight of the set of SV can't be greater than the weight of entire set
 - $2^kd < e^{2k/c}n$
 - $kd = O(d \log n)$ iterations

Analysis of Clarkson 2

- Synopsis:
 - $O(d \log n)$ iterations.
 - Each iteration requires $O(dn) + P(d^2, d)$ work
 - Total: $O(d^2n \log n) + d \log n P(d^2)$
- Combined with C1, this gives
 - $P_{C_2}(n) = d^2n + d^2 \log n P(d^2)$